

Technical Summary:

Complexity-Based Prompting for Multi-Step Reasoning

Kai-Yu Lu

1 Research Problem and Motivation

Large language models exhibit an emergent ability to perform multi-step reasoning when prompted with a few examples that contain explicit intermediate reasoning. Chain-of-thought (CoT) prompting provides such examples as short natural language steps that lead to a final answer. A central open problem is how to select the most effective examples and reasoning chains for in-context learning.

This paper studies the following core problem: given a multi-step reasoning task and a large language model, how should in-context examples and generated reasoning chains be selected so that the final answer accuracy is maximized, under limited annotation budget and without fine-tuning the model parameters. The work focuses on the notion of *reasoning complexity* and investigates whether examples with more reasoning steps are more effective both as prompts and as candidates during decoding.

The motivation arises from several limitations of prior practices. Existing CoT prompting often relies on small sets of manually written examples that are reused across tasks, which are difficult to generalize and do not provide a principled selection criterion. Retrieval-based selection over large annotated training sets can work well but requires substantial annotation effort, since every potential training instance needs a CoT explanation. Furthermore, expressive neural models are known to exploit superficial shortcuts and spurious correlations in training data, which can lead to shallow reasoning chains and poor generalization. The paper hypothesizes that explicitly preferring more complex reasoning examples and outputs can improve robustness and accuracy for multi-step reasoning tasks.

2 Related Work

2.1 Emergent Abilities and In-Context Learning

Scaling studies on language models have demonstrated that very large models develop emergent abilities that are absent in smaller models, including in-context learning and multi-step reasoning. In-context learning refers to the ability of a model to solve new tasks by conditioning on a small number of input-output pairs in the prompt, without parameter updates. Prior work shows that such abilities become prominent only at scales above approximately 100 billion parameters.

2.2 Chain-of-Thought Prompting and Self-Consistency

Chain-of-thought prompting introduces intermediate reasoning steps between the question and the answer. These steps are given as short sentences that decompose the problem into smaller subproblems. Previous work has shown that CoT prompting can significantly improve performance on math word problems and other reasoning tasks compared with direct answer prompting, where the model is asked to output only the final answer.

Self-consistency is an extension of CoT prompting at decoding time. Instead of generating a single reasoning chain using greedy decoding, the model is sampled multiple times to produce a set of possible reasoning chains and answers. A majority vote over the answers is then taken as the final prediction. This procedure can mitigate instability and reduce the impact of spurious reasoning paths.

2.3 Example Selection for Prompting

Example selection for prompting is known to be crucial yet difficult. Prior approaches include manual trial-and-error design, heuristic rules, automatic prompt search, and similarity-based retrieval from large training sets. Similarity-based retrieval typically requires a dense embedding model and substantial CoT annotation, since every training instance may be retrieved. For CoT prompting, this requirement drastically increases annotation cost and undermines the few-shot nature of the method.

The present paper proposes a different selection criterion based on reasoning complexity, defined primarily as the number of reasoning steps in a chain. This criterion is simple, task-aligned, and can be implemented with modest annotation effort.

3 Dataset Construction

3.1 Math Word Problem Datasets

The study evaluates the proposed methods on three math word problem datasets:

- **GSM8K**: A grade-school math benchmark consisting of natural language word problems with numeric answers. The dataset provides human-written CoT explanations for training instances. The test set contains approximately 1.3k examples. A validation split of 200 training examples is constructed for model selection.
- **MultiArith**: A dataset of arithmetic word problems involving basic operations such as addition, subtraction, multiplication, and division. The original dataset does not include CoT annotations. The test set contains 600 examples, and a validation set of 200 examples is sampled from the training data.
- **MathQA**: A dataset of math word problems with operation-based formalisms and multiple-choice answers. The dataset is known to contain noisy and sometimes inconsistent annotations. The test set contains 600 examples, with 200 training examples used as a validation set.

For GSM8K and MathQA, the existing training sets include human-annotated reasoning chains. For MultiArith, which lacks such chains, the authors approximate complexity by question length. The longest questions are selected and manually annotated with CoT explanations only for a small few-shot set, which keeps annotation cost low while still enabling complexity-based selection.

3.2 Non-Math Reasoning Datasets

Three non-math reasoning datasets are considered:

- **StrategyQA**: A commonsense question answering benchmark where each question requires implicit multi-step reasoning strategies rather than single facts. The test set contains 800 examples.
- **Date Understanding**: A temporal reasoning subset of the BigBench Hard suite, containing 250 test instances that require reasoning over dates and time intervals.

Dataset	Domain	Test Size	CoT in Training	Notes
GSM8K	Math word problems	1.3k	Yes	Clean annotations, numeric answers
MultiArith	Arithmetic word problems	600	No	CoT created for a few complex questions
MathQA	Math multiple-choice	600	Yes	Noisy labels, operation-based formalisms
StrategyQA	Commonsense QA	800	No	Implicit multi-step reasoning
Date Understanding	Temporal reasoning	250	No	Subset of BigBench Hard
Penguins	Referential reasoning	146	No	Subset of BigBench Hard, object comparisons

Table 1: Datasets used for evaluating complexity-based prompting and consistency.

- **Penguins:** A referential game subset of BigBench Hard with 146 test instances. Questions typically compare attributes of different entities, such as age relations between penguins.

These non-math datasets do not provide CoT annotations. Similar to MultiArith, question length is used as a proxy for complexity. The longest questions are selected for manual CoT annotation to form a small pool of complex examples used as prompts.

3.3 Summary of Dataset Properties

Table 1 summarizes the main properties of the datasets used in the study.

4 Query Protocol and Task Definitions

4.1 Task Definitions

All tasks considered in the paper are cast as supervised prediction problems conditioned on natural language input:

- **Math word problems** (GSM8K, MultiArith, MathQA): Given a natural language question describing a math scenario, the model must output either a numeric answer or a multiple-choice option.
- **Commonsense reasoning** (StrategyQA): Given a yes/no question that requires multi-step commonsense reasoning, the model outputs either “yes” or “no”.
- **Temporal reasoning** (Date Understanding): Given a description of dates, offsets, and events, the model outputs the correct date-related answer.
- **Referential reasoning** (Penguins): Given descriptions of entities and a question about their relations, the model outputs the correct answer comparing these entities.

The primary evaluation metric across all tasks is **accuracy**, defined as the fraction of test instances for which the predicted answer exactly matches the ground-truth label.

4.2 Prompting Protocol

The prompting protocol follows the chain-of-thought paradigm. For each dataset, a prompt is constructed as a concatenation of several CoT cases followed by a new test question:

1. Each CoT case consists of a question, a manually written reasoning chain composed of multiple short sentences, and a final answer.
2. A fixed number of cases is used in the prompt. For math datasets, eight cases are included. For non-math datasets, the number is chosen similarly for consistency.
3. After the last case, the test question is appended, followed by an instruction phrase such as “Let’s think step by step” that encourages the model to produce explicit reasoning.

At inference time, the model reads the entire prompt and generates a new reasoning chain and final answer for the test question.

4.3 Decoding Protocol and Self-Consistency

Two decoding regimes are considered:

- **Greedy decoding:** The model generates a single reasoning chain and answer by always selecting the most probable next token at each step.
- **Sampling with self-consistency:** The model is sampled multiple times for each test question, typically 50 samples. Each sample yields a reasoning chain and an answer. A majority vote is then taken over the answers, and the most frequent answer is chosen as the prediction.

The proposed complexity-based consistency modifies the self-consistency procedure by using only a subset of the sampled reasoning chains, chosen according to their complexity, as explained in Section 5.

5 Modeling Approach

The core contribution of the paper is a pair of selection schemes that prioritize complex reasoning chains at two stages of the reasoning pipeline:

- **Complexity-based prompting:** selection of in-context examples with complex reasoning chains for the input prompt.
- **Complexity-based consistency:** selection of complex generated reasoning chains during decoding for majority voting.

5.1 Definition of Reasoning Complexity

Reasoning complexity is defined primarily in terms of the **number of reasoning steps** in a chain. A step is treated as one line of text, separated by a newline character. Intuitively, more steps indicate a more detailed and potentially more cognitively demanding reasoning process.

When gold CoT annotations are available, the step count is computed directly from these annotations. When CoT annotations are absent, the paper uses alternative complexity proxies, such as question length or underlying formula length, to identify complex instances. For such datasets, only a small subset of complex instances is manually annotated with CoT explanations.

5.2 Complexity-Based Prompting

Complexity-based prompting selects in-context examples that maximize reasoning complexity as measured by step count. The procedure can be summarized as follows.

Step 1: Candidate pool construction For each dataset, a pool of candidate training instances with CoT annotations is assembled.

- If gold CoT explanations are present (GSM8K, MathQA), all training instances with chains are eligible.
- If CoT explanations are absent (MultiArith, non-math datasets), question length or formula length is used as a proxy measure. Only the instances with the highest proxy complexity are manually annotated to form the candidate pool.

Step 2: Complexity scoring Each candidate instance is assigned a complexity score equal to the number of reasoning steps in its CoT chain. This score is derived by splitting the chain into lines and counting them. Prompts with higher average number of steps per example are considered more complex.

Step 3: Prompt construction For each dataset, the final prompt is constructed by selecting the top- k instances with the largest complexity scores.

- For math datasets, eight complex instances are selected, each typically containing around nine reasoning steps on GSM8K.
- For comparison, two baselines are considered:
 - **Handcrafted CoT**: a fixed set of eight manually designed examples from prior work, not drawn from the dataset distribution.
 - **Random CoT**: eight random training instances with CoT chains drawn from the dataset.

The key design choice is to maximize the step count per example, rather than the total number of steps in the prompt or the raw character length of the prompt. The paper shows empirically that the per-example step count is the dominant factor for performance improvements.

5.3 Complexity-Based Consistency

Complexity-based consistency extends the complexity criterion to the output space during decoding. The procedure modifies self-consistency in the following way.

Step 1: Sampling multiple chains For each test question, the model is sampled N times, with $N = 50$ in the experiments. Each sample yields:

- A reasoning chain composed of multiple steps.
- A final answer extracted from the end of the chain.

Step 2: Output complexity scoring Each sampled chain is assigned a complexity score equal to its number of reasoning steps, computed in the same way as for input examples. The sampled chains are then sorted in descending order of step count.

Step 3: Voting over complex chains Instead of voting over all N chains, the method selects the top K chains with the largest step counts, where $K < N$ (typically between 30 and 40). A majority vote is then performed only over the answers from these complex chains. The most frequent answer within the top- K set is returned as the final prediction.

This procedure explicitly increases the weight of more complex reasoning paths and reduces the influence of short, potentially shortcut-like chains that may exploit superficial cues in the input. Experiments show that:

- Voting over the most complex chains outperforms voting over all chains.
- Voting over the simplest chains leads to lower accuracy than voting over all chains.

These observations provide evidence that complex reasoning chains are more reliable predictors of correct answers.

5.4 Alternative Complexity Proxies and Confounder Analysis

The paper also examines alternative complexity measures and potential confounding factors.

Alternative proxies When CoT annotations are unavailable at scale, complexity can be approximated using question length or formula length. The study shows that prompts constructed from questions with the longest lengths also lead to improved performance compared with prompts constructed from shorter questions. Similarly, prompts using instances with the longest underlying formulas (as defined in MathQA) yield better results than those with shorter formulas. These findings indicate that the trend of “more complex prompts yield better performance” is robust across several proxies.

Confounders Three major confounders are investigated:

1. **Total number of reasoning steps** in the prompt.
2. **Total prompt length** in characters.
3. **Per-step length** in characters.

The analysis constructs prompts that control for these factors. For example:

- A prompt with 24 simple examples of 3-step chains has the same total number of steps as a prompt with 8 complex examples of 9-step chains, but the complex prompt yields higher accuracy.
- A prompt that maximizes total character length does not outperform the prompt that maximizes per-example reasoning steps.
- Prompts with very short per-step text underperform those with moderate per-step length, even when the total number of steps is fixed.

These results show that the number of reasoning steps per example, given moderate per-step length, is the main driver of performance gains rather than trivial confounders.

Prompting scheme (Codex)	GSM8K Acc.	MultiArith Acc.	MathQA Acc.
Handcrafted CoT, greedy	61.0	95.8	29.3
Random CoT, greedy	60.4	97.3	40.5
Complex CoT, greedy	66.6	95.8	47.3
Handcrafted CoT, self-consistency	74.6	99.7	55.0
Random CoT, self-consistency	77.3	99.3	58.2
Complex CoT, self-consistency	82.6	99.7	58.6
Complex CoT + complex voting	82.9	99.8	60.0

Table 2: Accuracy on math benchmarks for code-davinci-002 under different prompting and voting schemes.

6 Empirical Results

6.1 Experimental Setup

Language models The experiments primarily use two publicly accessible models from the GPT-3 family:

- **text-davinci-002**: a general-purpose 175B parameter GPT-3 model.
- **code-davinci-002** (Codex): a 175B parameter model trained extensively on code, known to perform well on math and logic tasks.

For comparison, the paper also reports numbers from larger proprietary models:

- **LaMDA** (137B) and **PaLM** (540B).
- **Minerva** (540B), a math-focused model trained on LaTeX-style data.

In addition, fine-tuned baselines such as verifiers and custom sequence-to-sequence models from earlier work are included for context.

Decoding settings For math datasets, prompts always contain eight CoT cases. All prompting schemes append the phrase “Let’s think step by step” before the reasoning chain to strengthen CoT behavior. For self-consistency, 50 reasoning chains per test instance are sampled. The value of K for complexity-based consistency is tuned on validation sets and typically lies between 30 and 40.

6.2 Main Quantitative Results

6.2.1 Math word problem benchmarks

Table 2 summarizes the main results on GSM8K, MultiArith, and MathQA for code-davinci-002.

The results indicate several important findings:

- Under greedy decoding, complexity-based prompting improves GSM8K accuracy from 61.0 to 66.6 and MathQA accuracy from 29.3 to 47.3. On MathQA, the gain reaches 18 absolute points.
- Under self-consistency, complex prompts further improve over random and handcrafted prompts. On GSM8K, accuracy reaches 82.6 before complexity-based voting.

- Complexity-based consistency provides an additional gain, pushing GSM8K accuracy to 82.9 and MathQA accuracy to 60.0, establishing new state-of-the-art performance for models of this scale.

On MultiArith, performance is already near saturation, so the gains from complexity-based methods are small but do not degrade performance.

6.2.2 Non-math reasoning tasks

For StrategyQA, Date Understanding, and Penguins, complex prompts consistently outperform simpler ones on both text-davinci-002 and code-davinci-002. Notably:

- On StrategyQA, complex prompts with text-davinci-002 reach an accuracy comparable to or better than PaLM 540B, despite the smaller model size.
- On Date Understanding and Penguins, complex prompts with Codex achieve new state-of-the-art results, surpassing PaLM-based CoT prompting.

These outcomes show that the benefits of complexity-based prompting extend beyond math to various forms of symbolic and commonsense reasoning.

6.3 Ablation and Analysis

6.3.1 Contribution breakdown

An ablation study on GSM8K validation data decomposes the sources of improvement, including:

- Adding the “Let’s think step by step” phrase and adjusting prompt formatting.
- Replacing handcrafted prompts with complex prompts.
- Applying complexity-based consistency at decoding time.

More than half of the total accuracy gain over the original CoT baseline can be attributed directly to complexity-based prompting and complexity-based voting, indicating that the proposed mechanisms are major contributors rather than minor engineering tweaks.

6.3.2 Prompt distribution and robustness

The paper evaluates complex prompts under three prompt distribution conditions:

- **In-distribution:** prompts drawn from the same clean training data as the test distribution (GSM8K).
- **Noisy labels:** prompts drawn from a dataset with noisy annotations (MathQA).
- **Cross-dataset transfer:** prompts for one dataset (MultiArith) constructed using complex examples drawn from another dataset (GSM8K).

In all three settings, prompts constructed from more complex examples achieve higher or comparable accuracy to simpler prompts. The trend of increased accuracy with increased average reasoning steps holds consistently, except for a single minor exception in the transfer setting.

6.3.3 Comparison with other example selection schemes

Complexity-based selection is compared with several alternative selection strategies:

- **Random:** random choice of few-shot examples.
- **Centroid-based:** selection of examples whose sentence embeddings are closest to the dataset centroid, representing “typical” instances.
- **Retrieval-based:** selection of nearest neighbors to each test question in embedding space, requiring CoT annotation for the full training set.

On GSM8K and MultiArith, complexity-based selection outperforms all baselines while requiring only eight annotated examples. On MathQA, retrieval-based selection achieves the highest accuracy but at the cost of full-training-set CoT annotation and dependence on strong similarity between training and test distributions. Complexity-based selection offers a more attractive trade-off between performance and annotation cost.

6.3.4 Direction of generalization

An analysis of performance across test instances grouped by gold reasoning step count reveals an interesting pattern:

- On GSM8K and MathQA, complex prompts improve accuracy primarily on simpler test instances with fewer reasoning steps.
- On more complex instances, complex and simple prompts perform similarly.

This suggests that training the model with complex prompts yields reasoning capabilities that transfer downwards to simpler cases, rather than only benefiting the most difficult problems.

6.3.5 Model scale and emergence

Experiments with smaller models, such as text-curie-001 and Flan-T5 11B, show negligible or inconsistent gains from complexity-based prompting. In some cases, complex prompts slightly reduce accuracy. This indicates that the benefits of complexity-based selection are themselves emergent and require a sufficiently large underlying language model.

6.3.6 Step formatting and robustness

The study also tests robustness to step formatting by altering the delimiters between reasoning steps (new-line, period, semicolon, explicit “step i” markers). Although absolute accuracy varies with formatting, complex prompts consistently outperform simple prompts under all formats, reinforcing the stability of the complexity effect.

7 Summary

This paper introduces a complexity-based framework for in-context learning of multi-step reasoning with large language models. The central idea is to exploit the number of reasoning steps in chain-of-thought explanations as a simple yet powerful signal for both input example selection and output voting.

The main contributions can be summarized as follows:

- **Complexity-based prompting:** A prompting strategy that selects few-shot examples with the most complex reasoning chains, measured by step count, leading to substantial accuracy gains on multiple math and non-math reasoning benchmarks.
- **Complexity-based consistency:** A decoding strategy that performs majority voting only over the most complex generated reasoning chains, further improving performance over standard self-consistency.
- **Systematic empirical analysis:** A detailed investigation showing that per-example reasoning step count is the dominant factor, robust across datasets, complexity proxies, prompt perturbations, and distribution shifts, and that benefits emerge primarily in large models.

Limitations and Future Directions

The study also reveals several limitations and potential future directions:

- **Crude complexity measure:** Reasoning complexity is approximated by step count, which does not distinguish between truly informative steps and verbose but uninformative text. Future work could explore richer structural or semantic measures of complexity.
- **Task and domain coverage:** The evaluation focuses on math word problems and a small number of symbolic reasoning tasks. Extending the analysis to open-domain question answering, long-horizon planning, and dialogue would test the generality of the approach.
- **Model scalability:** The method is most effective for large models and does not provide clear gains for smaller models. Research on making complexity-based prompting beneficial for resource-constrained models would increase practical applicability.
- **Annotation efficiency:** Although the method requires only a small number of annotated CoT examples, obtaining high-quality reasoning chains in specialized domains such as medicine or law may still be expensive. Semi-automatic or verifier-assisted annotation may help reduce this cost.
- **Integration with verification:** Combining complexity-based selection with external verifiers or symbolic checkers could yield more reliable reasoning systems that not only prefer complex explanations but also verify their correctness.

Overall, the paper demonstrates that a simple principle, namely “prefer more complex reasoning chains,” can serve as an effective and annotation-efficient guideline for prompting large language models to perform multi-step reasoning. This principle offers a practical tool for designing prompts and decoding strategies in future research on reasoning with large language models.